DBMS ASSIGNMENT SERIES

Assignment 3

Database Design Analysis

Online Food Delivery System

SUBMITTED BY	PROGRAMME & INSTITUTION	SUBMISSION DATE
Vimal Kumar Rao	BCA (AI/ML) Alliance University	24 October 2026

A consistent relational design carried forward from Assignments 1 and 2.

IN THIS ASSIGNMENT

01	02	03	04
Review model	Trace dependencies	Find anomalies	Recommend fixes

1. Objective and System Scope

This report examines the Online Food Delivery System developed for Assignments 1 and 2. The database manages customer accounts and addresses, restaurant menus, delivery partners, customer orders, order line items, and payment status. The design uses eight related tables and focuses on minimizing duplication while preserving order-time prices and reliable references.

2. Existing Tables and Roles

Table	Purpose	Key fields / relationships
Customer	Account and contact details	PK Customer_ID; Email and Phone are UNIQUE
Address	Delivery addresses owned by customers	PK Address_ID; FK Customer_ID
Restaurant	Restaurant profile and rating	PK Restaurant_ID
Menu_Item	Restaurant food catalogue	PK Item_ID; FK Restaurant_ID
Delivery_Partner	Delivery worker and vehicle details	PK Delivery_ID; Phone and Vehicle_Number are UNIQUE
Orders	Order header and lifecycle	PK Order_ID; FKs Customer_ID, Restaurant_ID, Address_ID, Delivery_ID
Order_Item	Items and quantity for each order	Composite PK (Order_ID, Item_ID); both columns are FKs
Payment	Payment method, status, and date	PK Payment_ID; UNIQUE FK Order_ID

3. Redundancy Review

The current eight-table structure already removes many common duplication problems. The review found two avoidable derived-value risks in the earlier Assignment 2 schema and a few business rules worth enforcing explicitly.

Potential redundancy / risk	Why it matters	Improvement used in the revised design
Orders.Total_Amount	It can be recalculated from SUM(Quantity × Unit_Price). A stored value could disagree with the order lines after edits.	Remove the stored total for this coursework scope. Calculate the line-item subtotal in queries.
Payment.Amount	Under the stated one-payment-per-order model, this duplicates the same order subtotal and can drift from it.	Remove Payment.Amount; Payment references Orders and stores only method, status, and date.
Customer or restaurant details copied into every order line	A flattened spreadsheet repeats names, emails, restaurant names, and addresses.	Store each fact once in its own table and reference it by key.
Current menu price vs Order_Item.Unit_Price	At first glance these look duplicated, but the order line price records what the customer agreed to pay when ordering.	Keep Unit_Price as a historical transaction snapshot; do not recalculate old orders using today's menu price.

Potential redundancy / risk	Why it matters	Improvement used in the revised design
Plaintext password field	A database leak could expose credentials if raw passwords are stored.	Use Password_Hash populated by a secure password-hashing library. Sample hashes in this project are placeholders only.
Order address ownership	A valid Address_ID may still belong to a different customer if only a simple FK is used.	Use a composite FK from Orders(Address_ID, Customer_ID) to Address(Address_ID, Customer_ID).

4. Insertion, Deletion, and Update Anomalies

Insertion anomaly

In a single wide table that stores customer, restaurant, menu item, order, and payment details on every line, a restaurant cannot be recorded until at least one order line exists. The normalized design allows a Restaurant and its Menu_Item rows to be added independently of any order.

Deletion anomaly

If the final order line for a menu item is deleted from a flat table, that menu item's description and price may disappear with it. In the normalized design, deleting an Order_Item removes only the line; Menu_Item remains an independent restaurant catalogue record. Production systems should generally preserve completed order history rather than hard-delete it.

Update anomaly

If a restaurant address is copied into 200 menu/order rows, updating only 199 of them leaves conflicting addresses. Here, Restaurant.Address is stored once in Restaurant and Menu_Item carries only Restaurant_ID, so a restaurant address change is made in one place.

5. Functional Dependencies

Relation	Functional dependency	Reason
Customer	Customer_ID → Name, Email, Phone, Password_Hash	Customer_ID identifies one customer; unique Email and Phone are alternate identifiers.
Address	Address_ID → Customer_ID, House_No, Street, City, Pincode	One address row represents one saved location.
Restaurant	Restaurant_ID → Restaurant_Name, Phone, Address, Rating	Restaurant_ID identifies the restaurant profile.
Menu_Item	Item_ID → Restaurant_ID, Item_Name, Description, Price, Availability	Item_ID identifies one catalogue item.
Delivery_Partner	Delivery_ID → Name, Phone, Vehicle_Number, Status	Delivery_ID identifies one delivery partner.
Orders	Order_ID → Customer_ID, Restaurant_ID, Address_ID, Delivery_ID, Order_Date, Order_Status	Order_ID identifies one order header.
Order_Item	(Order_ID, Item_ID) → Quantity, Unit_Price	The full composite key identifies one menu item line in one order.
Payment	Payment_ID → Order_ID, Payment_Method, Payment_Status, Payment_Date; Order_ID → payment attributes	UNIQUE(Order_ID) allows at most one payment record per order.

6. Normalization Assessment

Normal form	Application to this design
First Normal Form (1NF)	Each attribute stores a single value. Multiple menu items are separate Order_Item rows rather than comma-separated values in Orders.
Second Normal Form (2NF)	Order_Item has a composite key (Order_ID, Item_ID); Quantity and Unit_Price depend on the full line key, not only on Order_ID or Item_ID.
Third Normal Form (3NF)	Customer, restaurant, menu item, and delivery-partner descriptions are stored with their own keys instead of depending indirectly on an order key.

7. Revised Design Decisions

- Remove Orders.Total_Amount and Payment.Amount from the coursework schema; compute the line-item subtotal using SUM(Quantity * Unit_Price). This scope assumes no taxes, discounts, or delivery fee. A production system with those charges should model them explicitly and snapshot finalized amounts for auditing.
- Rename Password to Password_Hash and store only strong, salted password hashes generated by the application.
- Keep Unit_Price in Order_Item so changes to current Menu_Item.Price do not rewrite historical order values.

- Use primary keys, foreign keys, unique constraints, check constraints, defaults, and indexes to block invalid states as early as possible.
- Use a composite foreign key to ensure an order can only use an address belonging to the same Customer_ID.
- Use soft deletion or retention rules for completed orders/payments in a real deployment, even though the coursework schema demonstrates dependent-row cascades for order line items.

8. Conclusion

The initial ER model has a sound normalized foundation. The main improvements are to avoid duplicated derived totals, protect credentials, enforce address ownership, and formalize value ranges/statuses. These decisions are carried forward consistently into Assignments 5–9.